

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

Practical No : 9

Aim :

- 9. A. linear probing (hash)
- 9. B. Write a python program to implement Breadth First Tree using Graph
- 9. C. Write a python program to implement Depth First Tree using Graph

CODE 9 A :

import matplotlib.pyplot as plt

```
def linear_probing_insert(hash_size, data):
    hash_table = [None] * hash_size

    for value in data:
        hash_index = value % hash_size
        original_index = hash_index

        while hash_table[hash_index] is not None:
            hash_index = (hash_index + 1) % hash_size

        if hash_index == original_index:
            print("Hash table is full!")
            return hash_table

        hash_table[hash_index] = value

    return hash_table

hash_size = int(input("Enter size of hash table: "))
data_input = input("Om Wala S119 Enter data (space-separated numbers): ")
data = list(map(int, data_input.split()))

final_table = linear_probing_insert(hash_size, data)

print("\nFinal Hash Table:")
for i, v in enumerate(final_table):
    print(f"Index {i}: {v}")

indices = list(range(hash_size))
values = [0 if v is None else v for v in final_table]

plt.figure(figsize=(10, 6))

plt.bar(indices, values)

plt.xlabel("Hash Table Index")
```

Name : Om Wala

Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
plt.ylabel("Stored Value")
plt.title("Linear Probing Hash Table")

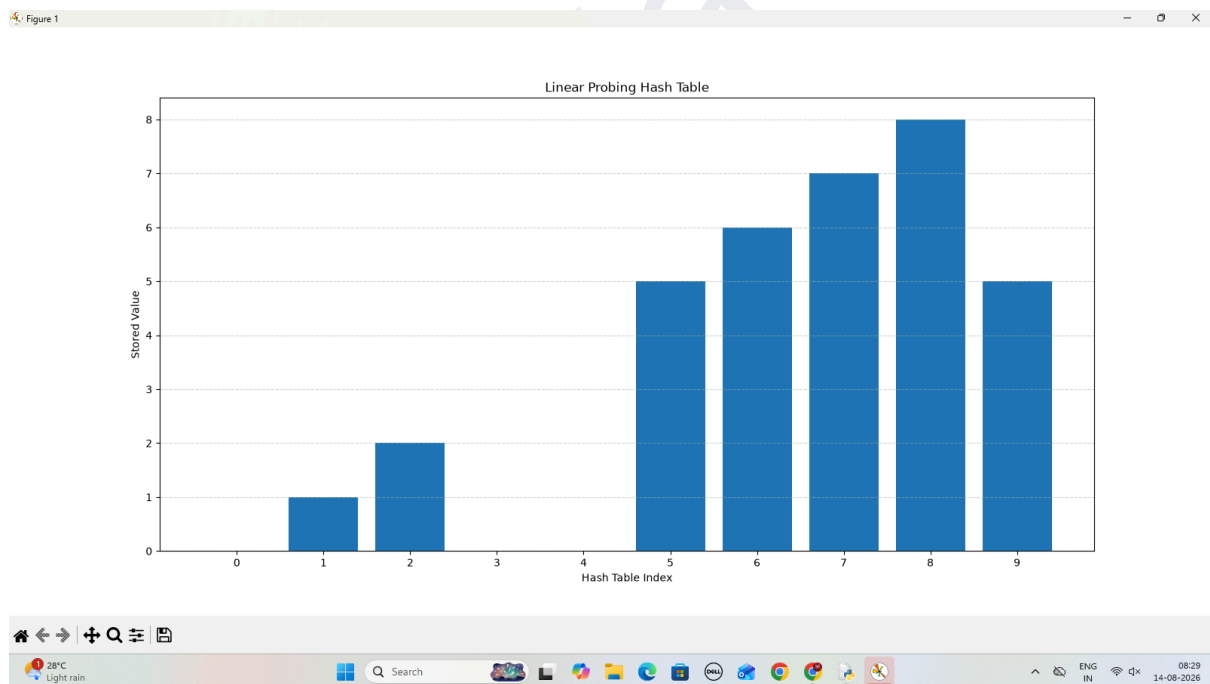
plt.xticks(indices)
plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.show()
```

OUTPUT :

```
Enter size of hash table: 10
Om Wala S119 Enter data (space-separated numbers): 0 8 5 6 2 7 1 5

Final Hash Table:
Index 0: 0
Index 1: 1
Index 2: 2
Index 3: None
Index 4: None
Index 5: 5
Index 6: 6
Index 7: 7
Index 8: 8
Index 9: 5
```



CODE 9A GUI :

```
import tkinter as tk
from tkinter import messagebox
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
def linear_probing_insert(hash_size, data):
    hash_table = [None] * hash_size

    for value in data:
        hash_index = value % hash_size
        original_index = hash_index

        while hash_table[hash_index] is not None:
            hash_index = (hash_index + 1) % hash_size

        if hash_index == original_index:
            messagebox.showerror("Error", "Hash table is full!")
            return hash_table

        hash_table[hash_index] = value

    return hash_table

def generate_table():
    try:
        hash_size = int(size_entry.get())

        if hash_size <= 0:
            messagebox.showerror("Error", "Enter a valid hash table size.")
            return

        data_input = data_entry.get()

        if not data_input:
            messagebox.showerror("Error", "Enter some data.")
            return

        data = list(map(int, data_input.split()))

        final_table = linear_probing_insert(hash_size, data)

        output_text.delete("1.0", tk.END)

        output_text.insert(tk.END, "Final Hash Table\n")
        output_text.insert(tk.END, "-" * 30 + "\n")

        for i, v in enumerate(final_table):
            output_text.insert(tk.END, f"Index {i}: {v}\n")

        for widget in graph_frame.winfo_children():
            widget.destroy()
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
indices = list(range(hash_size))
values = [0 if v is None else v for v in final_table]

figure = Figure(figsize=(8, 4.5), dpi=100)
axis = figure.add_subplot(111)

axis.bar(indices, values)

axis.set_title("Linear Probing Hash Table")
axis.set_xlabel("Hash Table Index")
axis.set_ylabel("Stored Value")

axis.set_xticks(indices)
axis.grid(axis="y", linestyle="--", alpha=0.5)

figure.tight_layout()

canvas = FigureCanvasTkAgg(figure, master=graph_frame)
canvas.draw()
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

except ValueError:
    messagebox.showerror(
        "Invalid Input",
        "Please enter only numbers separated by spaces."
    )

root = tk.Tk()
root.title("Linear Probing Hash Table - Om Wala S119")
root.geometry("1000x750")
root.configure(bg="#F4F6F8")

title_label = tk.Label(
    root,
    text="Linear Probing Hash Table",
    font=("Segoe UI", 24, "bold"),
    bg="#F4F6F8",
    fg="#333333"
)
title_label.pack(pady=(20, 5))

student_label = tk.Label(
    root,
    text="Om Wala S119",
    font=("Segoe UI", 14, "bold"),
    bg="#F4F6F8",
    fg="#4A90E2"
)
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
student_label.pack(pady=(0, 15))

input_frame = tk.Frame(
    root,
    bg="white",
    padx=20,
    pady=15
)
input_frame.pack(fill=tk.X, padx=30, pady=5)

size_label = tk.Label(
    input_frame,
    text="Hash Table Size:",
    font=("Segoe UI", 12, "bold"),
    bg="white"
)
size_label.grid(row=0, column=0, padx=10, pady=8)

size_entry = tk.Entry(
    input_frame,
    font=("Segoe UI", 12),
    width=15
)
size_entry.grid(row=0, column=1, padx=10, pady=8)

data_label = tk.Label(
    input_frame,
    text="Enter Data:",
    font=("Segoe UI", 12, "bold"),
    bg="white"
)
data_label.grid(row=1, column=0, padx=10, pady=8)

data_entry = tk.Entry(
    input_frame,
    font=("Segoe UI", 12),
    width=50
)
data_entry.grid(row=1, column=1, padx=10, pady=8)

generate_button = tk.Button(
    input_frame,
    text="Generate Hash Table",
    font=("Segoe UI", 12, "bold"),
    bg="#4A90E2",
    fg="white",
    padx=15,
    pady=8,
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
        command=generate_table
    )
    generate_button.grid(row=2, column=0, columnspan=2, pady=12)

    output_frame = tk.Frame(
        root,
        bg="white"
    )
    output_frame.pack(fill=tk.X, padx=30, pady=10)

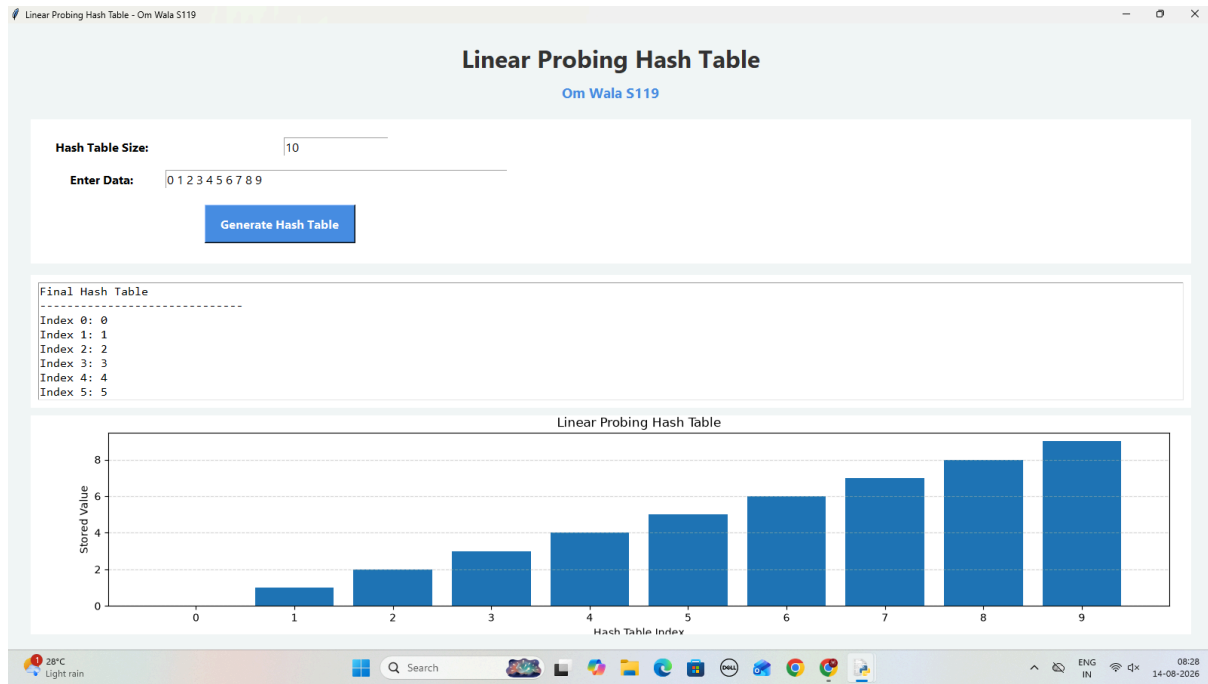
    output_text = tk.Text(
        output_frame,
        height=8,
        font=("Consolas", 12),
        bg="white"
    )
    output_text.pack(fill=tk.X, padx=10, pady=10)

    graph_frame = tk.Frame(
        root,
        bg="white"
    )
    graph_frame.pack(
        fill=tk.BOTH,
        expand=True,
        padx=30,
        pady=(0, 20)
    )

    root.mainloop()
```

OUTPUT :

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE



CODE 9B :

```
from collections import deque, defaultdict
import matplotlib.pyplot as plt
import networkx as nx
```

```
class Graph:
```

```
    def __init__(self):
        self.graph = defaultdict(list)
```

```
    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []
```

```
    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)
```

```
    def bfs_tree(self, start):
        visited = set()
        bfs_tree = defaultdict(list)
        queue = deque([start])
        visited.add(start)
```

```
        while queue:
            current = queue.popleft()
```

```
            for neighbor in self.graph[current]:
```

Name : Om Wala

Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
        if neighbor not in visited:
            visited.add(neighbor)
            bfs_tree[current].append(neighbor)
            queue.append(neighbor)

    return bfs_tree

def display(self):
    for vertex, edges in self.graph.items():
        print(f"{vertex}: {edges}")

def print_bfs_tree(bfs_tree, start):
    print(f"Breadth-First Tree starting from {start}:")

    for vertex in bfs_tree:
        print(f"{vertex}: {bfs_tree[vertex]}")

def plot_graph(graph, bfs_tree):
    G = nx.Graph()

    for vertex in graph.graph:
        G.add_node(vertex)

    for vertex in graph.graph:
        for neighbor in graph.graph[vertex]:
            G.add_edge(vertex, neighbor)

    position = nx.spring_layout(G, seed=42)

    plt.figure(figsize=(8, 6))

    nx.draw(
        G,
        position,
        with_labels=True,
        node_size=2000,
        font_size=14,
        font_weight="bold"
    )

    plt.title("BFS Graph - Om Wala S119")
    plt.show()

if __name__ == "__main__":
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
print("Om Wala S119")
print("=" * 30)

g = Graph()

for v in ['A', 'B', 'C', 'D', 'E', 'F']:
    g.add_vertex(v)

g.add_edge('A', 'B')
g.add_edge('A', 'C')
g.add_edge('B', 'D')
g.add_edge('B', 'E')
g.add_edge('C', 'F')

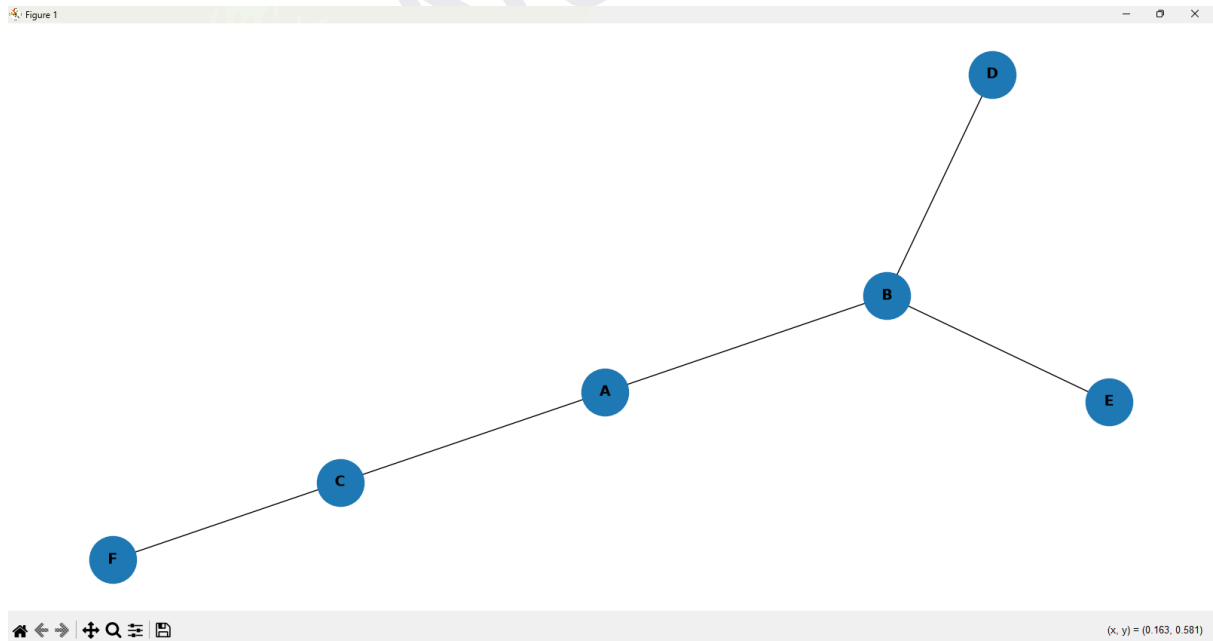
print("\nGraph:")
g.display()

start_vertex = 'A'
bfs_tree = g.bfs_tree(start_vertex)

print()
print_bfs_tree(bfs_tree, start_vertex)

plot_graph(g, bfs_tree)
```

OUTPUT :



CODE 9B GUI :

```
from collections import deque, defaultdict
import tkinter as tk
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
from tkinter import messagebox
import networkx as nx
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

```
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)

    def bfs_tree(self, start):
        visited = set()
        bfs_tree = defaultdict(list)
        queue = deque([start])
        visited.add(start)

        while queue:
            current = queue.popleft()

            for neighbor in self.graph[current]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    bfs_tree[current].append(neighbor)
                    queue.append(neighbor)

        return bfs_tree

    def display(self):
        result = ""

        for vertex, edges in self.graph.items():
            result += f"{vertex}: {edges}\n"

        return result

def create_graph():
    global graph
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
graph = Graph()

for vertex in ['A', 'B', 'C', 'D', 'E', 'F']:
    graph.add_vertex(vertex)

graph.add_edge('A', 'B')
graph.add_edge('A', 'C')
graph.add_edge('B', 'D')
graph.add_edge('B', 'E')
graph.add_edge('C', 'F')

def run_bfs():
    start = start_entry.get().upper()

    if start not in graph.graph:
        messagebox.showerror("Error", "Enter a valid starting vertex.")
        return

    bfs_tree = graph.bfs_tree(start)

    output_text.delete("1.0", tk.END)

    output_text.insert(tk.END, "Om Wala S119\n")
    output_text.insert(tk.END, "=" * 35 + "\n\n")

    output_text.insert(tk.END, "Graph:\n")
    output_text.insert(tk.END, graph.display())

    output_text.insert(
        tk.END,
        f"\nBreadth-First Tree starting from {start}:\n"
    )

    for vertex in bfs_tree:
        output_text.insert(
            tk.END,
            f"{vertex}: {bfs_tree[vertex]}\n"
        )

    draw_graph(start, bfs_tree)

def draw_graph(start, bfs_tree):
    for widget in graph_frame.winfo_children():
        widget.destroy()

    G = nx.Graph()
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
for vertex in graph.graph:
    G.add_node(vertex)

for vertex in graph.graph:
    for neighbor in graph.graph[vertex]:
        G.add_edge(vertex, neighbor)

position = {
    'A': (0, 2),
    'B': (-1, 1),
    'C': (1, 1),
    'D': (-2, 0),
    'E': (0, 0),
    'F': (2, 0)
}

figure = Figure(figsize=(7, 5), dpi=100)
axis = figure.add_subplot(111)

nx.draw(
    G,
    position,
    ax=axis,
    with_labels=True,
    node_size=1800,
    font_size=14,
    font_weight="bold"
)

axis.set_title(f"BFS Graph - Starting Vertex: {start}")

figure.tight_layout()

canvas = FigureCanvasTkAgg(
    figure,
    master=graph_frame
)

canvas.draw()
canvas.get_tk_widget().pack(
    fill=tk.BOTH,
    expand=True
)

root = tk.Tk()
root.title("BFS Graph - Om Wala S119")
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
root.geometry("1000x750")
root.configure(bg="#F4F6F8")

create_graph()

title_label = tk.Label(
    root,
    text="Breadth-First Search Graph",
    font=("Segoe UI", 24, "bold"),
    bg="#F4F6F8",
    fg="#333333"
)

title_label.pack(pady=(15, 5))

student_label = tk.Label(
    root,
    text="Om Wala S119",
    font=("Segoe UI", 14, "bold"),
    bg="#F4F6F8",
    fg="#4A90E2"
)

student_label.pack(pady=(0, 10))

input_frame = tk.Frame(
    root,
    bg="white",
    padx=20,
    pady=12
)

input_frame.pack(
    fill=tk.X,
    padx=30,
    pady=5
)

start_label = tk.Label(
    input_frame,
    text="Starting Vertex:",
    font=("Segoe UI", 12, "bold"),
    bg="white"
)

start_label.grid(
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
    row=0,  
    column=0,  
    padx=10,  
    pady=8  
)
```

```
start_entry = tk.Entry(  
    input_frame,  
    font=("Segoe UI", 12),  
    width=10  
)
```

```
start_entry.grid(  
    row=0,  
    column=1,  
    padx=10,  
    pady=8  
)
```

```
start_entry.insert(0, "A")
```

```
bfs_button = tk.Button(  
    input_frame,  
    text="Run BFS",  
    font=("Segoe UI", 12, "bold"),  
    bg="#4A90E2",  
    fg="white",  
    padx=20,  
    pady=7,  
    command=run_bfs  
)
```

```
bfs_button.grid(  
    row=0,  
    column=2,  
    padx=15,  
    pady=8  
)
```

```
output_frame = tk.Frame(  
    root,  
    bg="white"  
)
```

```
output_frame.pack()
```

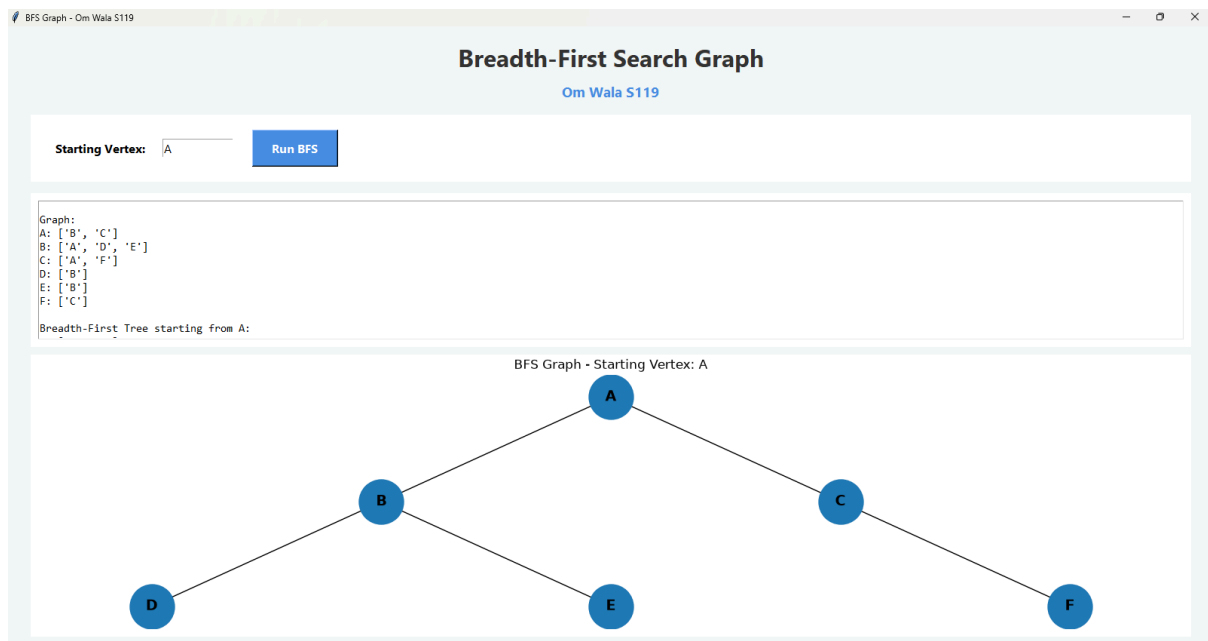
Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
        fill=tk.X,  
        padx=30,  
        pady=10  
    )  
  
    output_text = tk.Text(  
        output_frame,  
        height=10,  
        font=("Consolas", 11),  
        bg="white"  
    )  
  
    output_text.pack(  
        fill=tk.X,  
        padx=10,  
        pady=10  
    )  
  
    graph_frame = tk.Frame(  
        root,  
        bg="white"  
    )  
  
    graph_frame.pack(  
        fill=tk.BOTH,  
        expand=True,  
        padx=30,  
        pady=(0, 20)  
    )  
  
    root.mainloop()
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

OUTPUT :



CODE 9C :

```
import matplotlib.pyplot as plt
import networkx as nx
from collections import defaultdict
```

```
class Graph:
```

```
    def __init__(self):
        self.graph = defaultdict(list)
```

```
    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []
```

```
    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)
```

```
    def dfs_tree(self, start):
        """Perform DFS and return the Depth-First Tree as a dictionary."""
        visited = set()
        dfs_tree = defaultdict(list)
```

```
    def dfs(v):
        visited.add(v)
```

```
        for neighbor in self.graph[v]:
```

Name : Om Wala

Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
        if neighbor not in visited:
            dfs_tree[v].append(neighbor)
            dfs(neighbor)

    dfs(start)
    return dfs_tree

def display(self):
    for vertex, edges in self.graph.items():
        print(f"{vertex}: {edges}")

def visualize(self, dfs_tree=None):
    """Visualize the graph and optionally the DFS tree."""
    G = nx.Graph(self.graph)

    pos = nx.spring_layout(G)

    plt.figure(figsize=(12, 6))

    plt.subplot(121)

    nx.draw(
        G,
        pos,
        with_labels=True,
        node_color='lightblue',
        edge_color='gray',
        node_size=1000,
        font_size=15,
        font_weight='bold'
    )

    plt.title("Graph")

    if dfs_tree:
        T = nx.DiGraph(dfs_tree)

        pos_tree = nx.spring_layout(T)

        plt.subplot(122)

        nx.draw(
            T,
            pos_tree,
            with_labels=True,
            node_color='lightgreen',
            edge_color='blue',
            node_size=1000,
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
        font_size=15,
        font_weight='bold',
        arrows=True
    )

    plt.title("DFS Tree")

plt.show()

if __name__ == "__main__":
    g = Graph()

    for v in ['A', 'B', 'C', 'D', 'E', 'F']:
        g.add_vertex(v)

    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'D')
    g.add_edge('B', 'E')
    g.add_edge('C', 'F')

    print("Graph:")
    g.display()

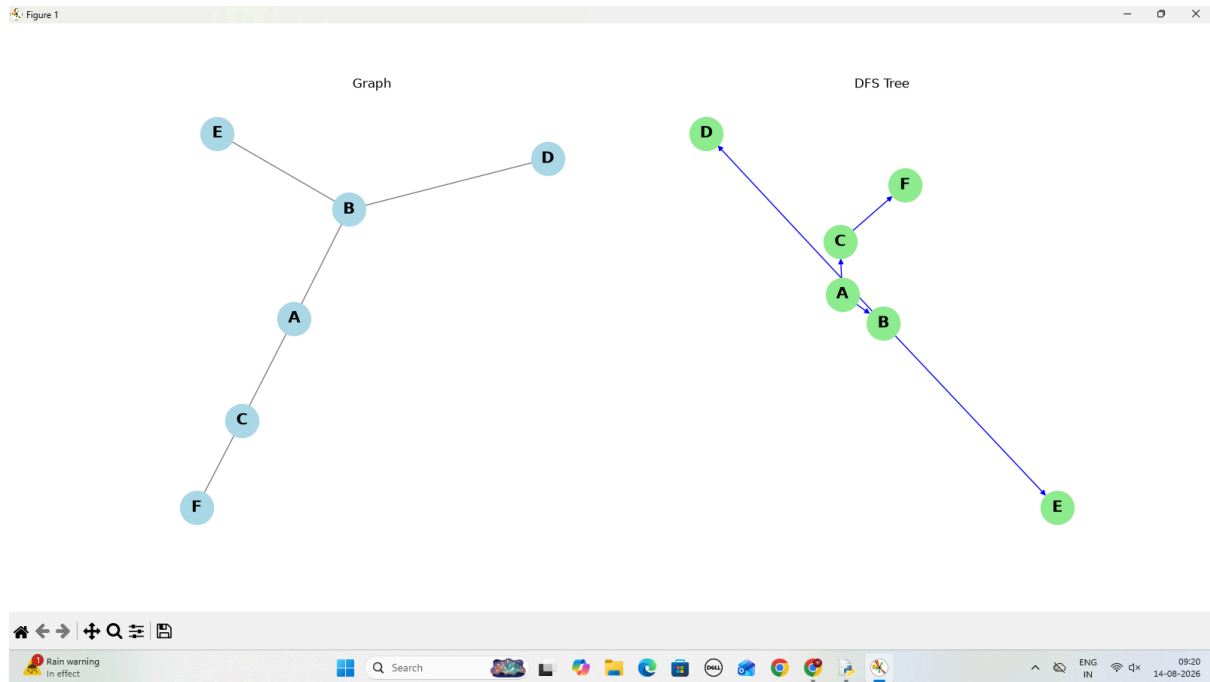
    start_vertex = 'A'
    dfs_tree = g.dfs_tree(start_vertex)

    g.visualize(dfs_tree)
```

OUTPUT :

```
===== RESTART: D:/Om Wala SYCS/Data structure/Practical9C.py =====
Graph:
A: ['B', 'C']
B: ['A', 'D', 'E']
C: ['A', 'F']
D: ['B']
E: ['B']
F: ['C']
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE



CODE 9C GUI :

```
import tkinter as tk
from tkinter import messagebox
from collections import defaultdict
import networkx as nx
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

```
class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)

    def dfs_tree(self, start):
        visited = set()
        dfs_tree = defaultdict(list)

        def dfs(vertex):
            visited.add(vertex)
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
        for neighbor in self.graph[vertex]:
            if neighbor not in visited:
                dfs_tree[vertex].append(neighbor)
                dfs(neighbor)

    dfs(start)
    return dfs_tree

def display(self):
    result = ""

    for vertex, edges in self.graph.items():
        result += f"{vertex}: {edges}\n"

    return result

def create_graph():
    global graph

    vertices_input = vertex_entry.get().strip()
    edges_input = edge_entry.get().strip()

    if not vertices_input:
        messagebox.showerror("Error", "Please enter vertices.")
        return

    graph = Graph()

    vertices = vertices_input.split()

    for vertex in vertices:
        graph.add_vertex(vertex.upper())

    if edges_input:
        edges = edges_input.split(",")

        for edge in edges:
            parts = edge.strip().split()

            if len(parts) != 2:
                messagebox.showerror(
                    "Error",
                    "Enter edges in the format: A-B, B-C, C-D"
                )
                return

            vertex1 = parts[0].upper()
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
vertex2 = parts[1].upper()

if vertex1 not in graph.graph or vertex2 not in graph.graph:
    messagebox.showerror(
        "Error",
        f"Vertex {vertex1} or {vertex2} does not exist."
    )
    return

graph.add_edge(vertex1, vertex2)

output_text.delete("1.0", tk.END)

output_text.insert(tk.END, "Om Wala S119\n")
output_text.insert(tk.END, "=" * 40 + "\n\n")
output_text.insert(tk.END, "Graph Created Successfully!\n\n")
output_text.insert(tk.END, "Graph:\n")
output_text.insert(tk.END, graph.display())

start_entry.delete(0, tk.END)

draw_original_graph()

def run_dfs():
    if not graph.graph:
        messagebox.showerror(
            "Error",
            "Please create the graph first."
        )
        return

    start = start_entry.get().strip().upper()

    if not start:
        messagebox.showerror(
            "Error",
            "Please enter a starting vertex."
        )
        return

    if start not in graph.graph:
        messagebox.showerror(
            "Error",
            f"Vertex {start} does not exist."
        )
        return
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
dfs_tree = graph.dfs_tree(start)

output_text.delete("1.0", tk.END)

output_text.insert(tk.END, "Om Wala S119\n")
output_text.insert(tk.END, "=" * 40 + "\n\n")

output_text.insert(tk.END, "Graph:\n")
output_text.insert(tk.END, graph.display())

output_text.insert(
    tk.END,
    f"\nDFS Tree starting from {start}:\n"
)

for vertex in dfs_tree:
    if dfs_tree[vertex]:
        output_text.insert(
            tk.END,
            f"{vertex}: {dfs_tree[vertex]}\n"
        )

draw_graph(start, dfs_tree)

def draw_original_graph():
    for widget in graph_frame.winfo_children():
        widget.destroy()

G = nx.Graph()

for vertex in graph.graph:
    G.add_node(vertex)

for vertex in graph.graph:
    for neighbor in graph.graph[vertex]:
        G.add_edge(vertex, neighbor)

figure = Figure(figsize=(8, 4.5), dpi=100)
axis = figure.add_subplot(111)

position = nx.spring_layout(G, seed=42)

nx.draw(
    G,
    position,
    ax=axis,
    with_labels=True,
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
        node_size=1500,
        font_size=12,
        font_weight="bold"
    )

    axis.set_title("Original Graph")

    figure.tight_layout()

    canvas = FigureCanvasTkAgg(
        figure,
        master=graph_frame
    )

    canvas.draw()

    canvas.get_tk_widget().pack(
        fill=tk.BOTH,
        expand=True
    )

def draw_graph(start, dfs_tree):
    for widget in graph_frame.winfo_children():
        widget.destroy()

    G = nx.Graph()

    for vertex in graph.graph:
        G.add_node(vertex)

    for vertex in graph.graph:
        for neighbor in graph.graph[vertex]:
            G.add_edge(vertex, neighbor)

    T = nx.DiGraph()

    for vertex in dfs_tree:
        for neighbor in dfs_tree[vertex]:
            T.add_edge(vertex, neighbor)

    figure = Figure(figsize=(10, 5), dpi=100)

    graph_axis = figure.add_subplot(121)
    tree_axis = figure.add_subplot(122)

    position = nx.spring_layout(G, seed=42)
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
nx.draw(  
    G,  
    position,  
    ax=graph_axis,  
    with_labels=True,  
    node_size=1400,  
    font_size=11,  
    font_weight="bold"  
)  
  
graph_axis.set_title("Original Graph")  
  
tree_position = nx.spring_layout(T, seed=42)  
  
nx.draw(  
    T,  
    tree_position,  
    ax=tree_axis,  
    with_labels=True,  
    node_size=1400,  
    font_size=11,  
    font_weight="bold",  
    arrows=True  
)  
  
tree_axis.set_title("DFS Tree")  
  
figure.suptitle(  
    f'DFS Visualization - Starting Vertex: {start}'  
)  
  
figure.tight_layout()  
  
canvas = FigureCanvasTkAgg(  
    figure,  
    master=graph_frame  
)  
  
canvas.draw()  
  
canvas.get_tk_widget().pack(  
    fill=tk.BOTH,  
    expand=True  
)  
  
graph = Graph()
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
root = tk.Tk()
root.title("DFS Graph - Om Wala S119")
root.geometry("1100x800")
root.configure(bg="#F4F6F8")
```

```
title_label = tk.Label(
    root,
    text="Depth-First Search Graph",
    font=("Segoe UI", 24, "bold"),
    bg="#F4F6F8",
    fg="#333333"
)
```

```
title_label.pack(pady=(15, 3))
```

```
student_label = tk.Label(
    root,
    text="Om Wala S119",
    font=("Segoe UI", 14, "bold"),
    bg="#F4F6F8",
    fg="#4A90E2"
)
```

```
student_label.pack(pady=(0, 10))
```

```
input_frame = tk.Frame(
    root,
    bg="white",
    padx=20,
    pady=12
)
```

```
input_frame.pack(
    fill=tk.X,
    padx=30,
    pady=5
)
```

```
vertex_label = tk.Label(
    input_frame,
    text="Vertices:",
    font=("Segoe UI", 11, "bold"),
    bg="white"
)
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
vertex_label.grid(  
    row=0,  
    column=0,  
    padx=8,  
    pady=8  
)
```

```
vertex_entry = tk.Entry(  
    input_frame,  
    font=("Segoe UI", 11),  
    width=40  
)
```

```
vertex_entry.grid(  
    row=0,  
    column=1,  
    padx=8,  
    pady=8  
)
```

```
edge_label = tk.Label(  
    input_frame,  
    text="Edges:",  
    font=("Segoe UI", 11, "bold"),  
    bg="white"  
)
```

```
edge_label.grid(  
    row=1,  
    column=0,  
    padx=8,  
    pady=8  
)
```

```
edge_entry = tk.Entry(  
    input_frame,  
    font=("Segoe UI", 11),  
    width=40  
)
```

```
edge_entry.grid(  
    row=1,  
    column=1,  
    padx=8,
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
pady=8
)

create_button = tk.Button(
    input_frame,
    text="Create Graph",
    font=("Segoe UI", 11, "bold"),
    bg="#4A90E2",
    fg="white",
    padx=15,
    pady=6,
    command=create_graph
)

create_button.grid(
    row=0,
    column=2,
    rowspan=2,
    padx=15
)

start_label = tk.Label(
    input_frame,
    text="Starting Vertex:",
    font=("Segoe UI", 11, "bold"),
    bg="white"
)

start_label.grid(
    row=2,
    column=0,
    padx=8,
    pady=8
)

start_entry = tk.Entry(
    input_frame,
    font=("Segoe UI", 11),
    width=15
)

start_entry.grid(
    row=2,
    column=1,
    sticky="w",
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
    padx=8,  
    pady=8  
)  
  
dfs_button = tk.Button(  
    input_frame,  
    text="Run DFS",  
    font=("Segoe UI", 11, "bold"),  
    bg="#50C878",  
    fg="white",  
    padx=25,  
    pady=6,  
    command=run_dfs  
)  
  
dfs_button.grid(  
    row=2,  
    column=2,  
    padx=15  
)  
  
output_frame = tk.Frame(  
    root,  
    bg="white"  
)  
  
output_frame.pack(  
    fill=tk.X,  
    padx=30,  
    pady=10  
)  
  
output_text = tk.Text(  
    output_frame,  
    height=9,  
    font=("Consolas", 11),  
    bg="white"  
)  
  
output_text.pack(  
    fill=tk.X,  
    padx=10,  
    pady=10  
)
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : DATA STRUCTURE

```
graph_frame = tk.Frame(  
    root,  
    bg="white"  
)
```

```
graph_frame.pack(  
    fill=tk.BOTH,  
    expand=True,  
    padx=30,  
    pady=(0, 20)  
)
```

```
root.mainloop()
```

OUTPUT :

